

ThunderERP

Database Engineering Reference Guide

PostgreSQL + Prisma ORM

Intellectual Property of AB Global Traders

Stack: PostgreSQL 16 • Prisma ORM • NestJS • TypeScript

0. How to Use This Document

This is NOT a setup guide. This is a system-level data engineering handbook. The database layer is the foundation that every other part of ThunderERP depends on. If you get this layer wrong, you corrupt financial data, break inventory consistency, and cause irreversible system errors.



CRITICAL WARNING: Mistakes at the database layer are the hardest to fix. Unlike a bug in a service, a corrupted database record cannot be undone by a code push. Always understand what you are doing before writing to the database.

How to Read in Layers

- Day 1: Read Sections 0–2. Complete setup. Understand the Golden Data Flow.
- Week 1: Master Sections 3–4. Schema design, data types, and Prisma basics.
- Week 2: Study Sections 5–7. Relationships, constraints, and migrations.
- Week 3: Deep-dive Sections 8–10. Transactions, integrity, and advanced queries.
- Ongoing: Reference Sections 11–19 as needed for debugging, performance, and production.



KEY INSIGHT: You will not understand everything on first read. That is expected. Return to this document as you build. The concepts in Section 8 (Transactions) will make complete sense only after you have written your first order creation service.

1. System Overview — The Role of the Database

The database is the single source of truth for the entire ThunderERP system. Every other component — the frontend, the backend services, the APIs — is ultimately just a layer that reads from or writes to the database. If the database has bad data, everything built on top of it is broken.

1.1 The Three-Tier Responsibility Model

Tier	Technology	Responsibility
Database	PostgreSQL + Prisma	Store data. Enforce constraints. Guarantee integrity. Maintain consistency.
Backend	NestJS + TypeScript	Apply business logic. Validate input. Manage transactions. Talk to the DB.
Frontend	React	Display data. Capture user input. Call APIs. Never touch the DB directly.

 **STRICT RULE:** The database does NOT depend on assumptions from the frontend or backend. Constraints live IN the database. Even if someone bypasses your API and writes directly to the DB, constraints must still prevent invalid data.

1.2 The Golden Data Flow

Every data issue can be debugged by tracing this path. When something is wrong, start from the bottom (database) and work upward to the UI.

```
User Action (button click)
→ React UI sends HTTP request
→ NestJS Controller receives request
→ DTO validates input shape
→ Service applies business logic
→ Prisma ORM translates to SQL
→ PostgreSQL enforces constraints
→ Transaction commits (or rolls back)
→ Response returned up the chain
→ React UI updates
```

 **DEBUGGING RULE:** When data is wrong, start at the bottom — check the raw DB record first. Then check the Prisma query. Then check the service logic. Then check the DTO. Work upward, not downward.

1.3 Why PostgreSQL?

We chose PostgreSQL over other databases (MySQL, SQLite, MongoDB) for specific reasons that matter in an ERP context:

Feature	Why It Matters for ERP
ACID Transactions	Atomicity, Consistency, Isolation, Durability — essential for financial data.
Strong Data Types	Decimal type for money (no floating-point errors). Enum types for status fields.
Foreign Key Enforcement	Prevents orphan records. No order can reference a deleted customer.
JSON/JSONB Support	Store flexible metadata without schema changes — used for extensibility buffer.
Full Text Search	Built-in search across product names, customer records without Elasticsearch.
Row-Level Locking	Prevents race conditions on stock updates and financial records.
Schemas & Multi-tenancy	Separate schemas per tenant — foundational for SaaS ERP architecture.
Mature & Battle-tested	Used by Uber, Instagram, Apple. Handles millions of transactions per day.

2. Installation & Setup

2.1 Install PostgreSQL

Option A — Direct Installation (Recommended for Beginners)

1. Go to <https://www.postgresql.org/download/> and choose your OS.
2. Download and run the installer.
3. During installation: keep default port 5432, set a strong password for the postgres superuser (write it down), install pgAdmin 4.
4. Finish and restart your computer.

Option B — Docker (Recommended for Team Consistency)

Docker ensures every team member runs the exact same PostgreSQL version with the same settings. This eliminates "works on my machine" issues.

```
# docker-compose.yml
version: "3.8"
services:
  postgres:
    image: postgres:16-alpine
    container_name: thunder-postgres
    environment:
      POSTGRES_PASSWORD: YourStrongPassword123
      POSTGRES_DB: thunder_erp
      POSTGRES_USER: admin
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data # Persist data between restarts
    restart: unless-stopped

volumes:
  postgres_data:

# Start:
docker compose up -d

# Stop:
docker compose down

# Stop and DELETE all data:
docker compose down -v
```

2.2 Verify PostgreSQL Installation

```
# Connect via terminal:
psql -U postgres -h localhost
```

```
# Enter your password when prompted.
# Success: you will see the postgres=# prompt.

# Useful psql commands:
\l           -- List all databases
\c thunder_erp -- Connect to a database
\dt          -- List all tables
\d products  -- Describe table structure
\q          -- Quit
```

2.3 GUI Tools

pgAdmin 4 — For Server Administration

Installed automatically with PostgreSQL. Use it for: server monitoring, user management, viewing query execution plans, and backup/restore.

DBeaver — For Daily Development Work

Download from <https://dbeaver.io/download/> (free Community Edition). DBeaver is more powerful for daily queries, table browsing, and schema visualization.

- Create new connection → Database Type: PostgreSQL
- Host: localhost, Port: 5432, Database: thunder_erp
- Username: postgres (or your admin user), Password: your password
- Test Connection → Save

 **TIP:** Use DBeaver for browsing data and writing test queries. Use pgAdmin only for server administration tasks like creating users or monitoring performance.

2.4 Prisma ORM Setup in NestJS

```
# Inside your NestJS project folder:

# Step 1: Install Prisma packages
npm install prisma --save-dev
npm install @prisma/client

# Step 2: Initialize Prisma (creates prisma/schema.prisma and .env)
npx prisma init
```

2.5 Configure Database Connection

```
# .env file (NEVER commit this to git)
DATABASE_URL="postgresql://postgres:YourStrongPassword123@localhost:5432/thunder_erp?schema=public"

# Connection string format:
```

```
# postgresql://USER:PASSWORD@HOST:PORT/DATABASE?schema=SCHEMA
```

2.6 PrismaService — The NestJS Integration

Create a dedicated PrismaService that manages the database connection lifecycle. This is the class you inject into all your other services.

```
// src/prisma/prisma.service.ts
import { Injectable, OnModuleInit, OnModuleDestroy } from "@nestjs/common";
import { PrismaClient } from "@prisma/client";

@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit,
OnModuleDestroy {

  async onModuleInit() {
    await this.$connect();
  }

  async onModuleDestroy() {
    await this.$disconnect();
  }
}

// src/prisma/prisma.module.ts
@Module({
  providers: [PrismaService],
  exports: [PrismaService], // Export so other modules can inject it
})
export class PrismaModule {}

// Usage in any service:
constructor(private prisma: PrismaService) {}
```

2.7 First Migration

```
# After defining your schema, generate the Prisma client:
npx prisma generate

# Create and apply your first migration:
npx prisma migrate dev --name init

# Open the database browser:
npx prisma studio
```

⚠ CRITICAL RULE: Run "npx prisma generate" EVERY TIME you change schema.prisma. Without this, your TypeScript code will not know about the new fields and you will get runtime errors.

3. Prisma Schema Design — The Blueprint

The schema.prisma file is the single source of truth for your entire database structure. Every table, column, data type, relationship, and constraint is defined here. The database is generated FROM this file — never the other way around.

3.1 Schema File Structure

```
// prisma/schema.prisma

// 1. Generator - defines what Prisma generates
generator client {
  provider = "prisma-client-js"
}

// 2. Datasource - defines which database to connect to
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL") // Always use env vars, never hardcode
}

// 3. Models - define your tables
model Product {
  id      Int      @id @default(autoincrement())
  name    String
}
```

3.2 Mandatory Fields — ERP Standard

Every single model in ThunderERP MUST include these fields. No exceptions. They are required for auditing, soft deletes, and data traceability.

```
// Add these to EVERY model:
model AnyModel {
  id      Int      @id @default(autoincrement())

  // Soft delete - NEVER use DELETE
  isActive Boolean @default(true)
  deletedAt DateTime?
  deletedBy Int?

  // Audit trail - WHO and WHEN
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt // Auto-updated by Prisma on every update
  createdBy Int
  updatedBy Int?
}
```


3.3 Data Types — The Correct Choices for ERP

Choosing the wrong data type is a silent bug that causes corruption. These rules are non-negotiable.

What You Are Storing	Correct Prisma / PostgreSQL Type
Money, prices, tax amounts	Decimal @db.Decimal(12, 2) — NEVER use Float (see callout below)
Quantities (units, items)	Int — whole numbers only
Large quantities or IDs	BigInt — for high-volume systems
Status fields (PENDING, ACTIVE)	enum — never a plain String (enforces valid values)
Dates and times	DateTime — always stored as UTC
Short text (names, codes)	String — maps to VARCHAR(191) by default
Long text (notes, descriptions)	String @db.Text — no length limit
Flexible/custom fields	Json — use sparingly, hard to query efficiently
True/False flags	Boolean — isActive, isPaid, isVerified
Unique identifiers (alternate)	String @id @default(uuid()) — for distributed systems

 **FLOAT IS FORBIDDEN FOR MONEY:** Float stores 99.99 as 99.98999999999999. This is a binary representation problem. After enough calculations, your financial totals will be wrong by fractions of a rupee — and those errors compound. ALWAYS use Decimal @db.Decimal(12, 2) for any monetary value.

3.4 Enums — Enforcing Valid Status Values

Enums ensure status fields can only contain pre-defined values. Without enums, someone could accidentally set a status to "PENDINGG" (typo) and your state machine logic breaks.

```
// Define enums before the models that use them:
enum OrderStatus {
  DRAFT
  PENDING
  CONFIRMED
  SHIPPED
  DELIVERED
  COMPLETED
  CANCELLED
}

enum PaymentMethod {
  CASH
  UPI
  BANK_TRANSFER
  CHEQUE
}
```

```

enum StockMovementType {
    IN    // Stock received (purchase)
    OUT   // Stock sold (order)
    ADJUSTMENT // Manual correction
}

// Use in model:
model Order {
    status OrderStatus @default(DRAFT)
}

```

3.5 The Complete ThunderERP Schema

This is the full reference schema. Study every model, field, and relationship.

```

generator client {
    provider = "prisma-client-js"
}

datasource db {
    provider = "postgresql"
    url      = env("DATABASE_URL")
}

// ——— ENUMS ———
enum OrderStatus { DRAFT PENDING CONFIRMED SHIPPED DELIVERED COMPLETED CANCELLED }
enum InvoiceStatus { UNPAID PARTIALLY_PAID PAID OVERDUE CANCELLED }
enum PaymentMethod { CASH UPI BANK_TRANSFER CHEQUE }
enum MovementType { IN OUT ADJUSTMENT }
enum PlanType { BASIC PREMIUM ENTERPRISE }

// ——— TENANT (Multi-tenancy root) ———
model Tenant {
    id          Int          @id @default(autoincrement())
    name        String
    planType    PlanType     @default(BASIC)
    isActive    Boolean      @default(true)
    createdAt   DateTime     @default(now())
    updatedAt   DateTime     @updatedAt
    // Relations
    products    Product[]
    customers   Customer[]
    suppliers   Supplier[]
    orders      Order[]
    purchases   Purchase[]
    invoices    Invoice[]
    users       User[]
    @@map("tenants")
}

```

```
// ——— USER —————
model User {
    id            Int            @id @default(autoincrement())
    tenantId      Int
    tenant        Tenant        @relation(fields: [tenantId], references: [id])
    email         String        @unique
    passwordHash  String
    name          String
    role          String        @default("staff")
    isActive      Boolean        @default(true)
    createdAt     DateTime       @default(now())
    updatedAt     DateTime       @updatedAt
    @@map("users")
}

// ——— PRODUCT —————
model Product {
    id            Int            @id @default(autoincrement())
    tenantId      Int
    tenant        Tenant        @relation(fields: [tenantId], references: [id])
    name          String        @db.VarChar(200)
    sku           String?       @unique // Stock Keeping Unit code
    category      String?
    unit          String        @default("pcs")
    price         Decimal       @db.Decimal(12, 2)
    gstPercentage Decimal       @db.Decimal(5, 2) @default(18.00)
    metadata      Json?
    isActive      Boolean        @default(true)
    deletedAt     DateTime?
    deletedBy     Int?
    createdAt     DateTime       @default(now())
    updatedAt     DateTime       @updatedAt
    createdBy     Int
    updatedBy     Int?
    // Relations
    inventory     Inventory?
    orderItems    OrderItem[]
    purchaseItems PurchaseItem[]
    stockMovements StockMovement[]
    @@map("products")
}

// ——— INVENTORY —————
model Inventory {
    id            Int            @id @default(autoincrement())
    productId     Int            @unique // One-to-One with Product
    product       Product        @relation(fields: [productId], references: [id])
    availableQty  Int            @default(0)
    reservedQty   Int            @default(0)
    reorderLevel  Int            @default(10)
    updatedAt     DateTime       @updatedAt
    @@map("inventory")
}
```

```
// ——— CUSTOMER —————
model Customer {
  id          Int          @id @default(autoincrement())
  tenantId    Int
  tenant      Tenant      @relation(fields: [tenantId], references: [id])
  name        String
  phone       String?
  email       String?
  address     String?     @db.Text
  gstNumber   String?
  metadata    Json?
  isActive    Boolean     @default(true)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  createdBy   Int
  orders      Order[]
  @@map("customers")
}

// ——— SUPPLIER —————
model Supplier {
  id          Int          @id @default(autoincrement())
  tenantId    Int
  tenant      Tenant      @relation(fields: [tenantId], references: [id])
  name        String
  email       String?
  phone       String?
  gstNumber   String?
  isActive    Boolean     @default(true)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  createdBy   Int
  purchases   Purchase[]
  @@map("suppliers")
}

// ——— ORDER & ORDER ITEMS —————
model Order {
  id          Int          @id @default(autoincrement())
  tenantId    Int
  tenant      Tenant      @relation(fields: [tenantId], references: [id])
  customerId  Int
  customer    Customer    @relation(fields: [customerId], references: [id])
  orderDate   DateTime    @default(now())
  status      OrderStatus @default(DRAFT)
  totalAmount Decimal     @db.Decimal(12, 2)
  taxAmount   Decimal     @db.Decimal(12, 2) @default(0)
  notes       String?     @db.Text
  isActive    Boolean     @default(true)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  createdBy   Int
}
```

```

    updatedBy    Int?
    items        OrderItem[]
    invoice      Invoice?
    @@map("orders")
}

model OrderItem {
    id            Int            @id @default(autoincrement())
    orderId       Int
    order         Order          @relation(fields: [orderId], references: [id], onDelete:
Cascade)
    productId     Int
    product       Product        @relation(fields: [productId], references: [id])
    quantity      Int
    unitPrice     Decimal        @db.Decimal(12, 2) // Price AT TIME OF ORDER — never reference
current price
    gstPct        Decimal        @db.Decimal(5, 2)
    @@map("order_items")
}

// ——— PURCHASE & PURCHASE ITEMS ———
model Purchase {
    id            Int            @id @default(autoincrement())
    tenantId      Int
    tenant        Tenant         @relation(fields: [tenantId], references: [id])
    supplierId    Int
    supplier       Supplier       @relation(fields: [supplierId], references: [id])
    date          DateTime        @default(now())
    status        String          @default("PENDING")
    isActive      Boolean         @default(true)
    createdAt     DateTime        @default(now())
    updatedAt     DateTime        @updatedAt
    createdBy     Int
    items         PurchaseItem[]
    @@map("purchases")
}

model PurchaseItem {
    id            Int            @id @default(autoincrement())
    purchaseId    Int
    purchase       Purchase       @relation(fields: [purchaseId], references: [id], onDelete:
Cascade)
    productId     Int
    product       Product        @relation(fields: [productId], references: [id])
    quantity      Int
    unitPrice     Decimal        @db.Decimal(12, 2)
    @@map("purchase_items")
}

// ——— INVOICE & PAYMENT ———
model Invoice {
    id            Int            @id @default(autoincrement())
    tenantId      Int

```

```

    tenant      Tenant      @relation(fields: [tenantId], references: [id])
    orderId     Int          @unique // One-to-One with Order
    order       Order        @relation(fields: [orderId], references: [id])
    taxAmount   Decimal      @db.Decimal(12, 2)
    totalAmount Decimal      @db.Decimal(12, 2)
    status      InvoiceStatus @default(UNPAID)
    dueDate     DateTime?
    isActive    Boolean      @default(true)
    createdAt   DateTime      @default(now())
    updatedAt   DateTime      @updatedAt
    createdBy   Int
    payments    Payment[]
    @@map("invoices")
}

model Payment {
  id          Int          @id @default(autoincrement())
  tenantId    Int
  invoiceId   Int
  invoice     Invoice       @relation(fields: [invoiceId], references: [id])
  amount      Decimal      @db.Decimal(12, 2)
  method      PaymentMethod
  date        DateTime      @default(now())
  reference   String?      // Transaction ID, cheque number, etc.
  createdBy   Int
  transaction LedgerEntry?
  @@map("payments")
}

// ——— LEDGER (Double-Entry Accounting) ———
model LedgerEntry {
  id          Int          @id @default(autoincrement())
  paymentId   Int          @unique
  payment     Payment      @relation(fields: [paymentId], references: [id])
  type        String       // DEBIT or CREDIT
  amount      Decimal      @db.Decimal(12, 2)
  accountCode String       // e.g., "CASH", "ACCOUNTS_RECEIVABLE"
  entryDate   DateTime      @default(now())
  createdBy   Int
  @@map("ledger_entries")
}

// ——— STOCK MOVEMENT (Full Audit Trail) ———
model StockMovement {
  id          Int          @id @default(autoincrement())
  productId   Int
  product     Product      @relation(fields: [productId], references: [id])
  quantity    Int
  type        MovementType
  referenceType String      // "ORDER" or "PURCHASE" or "ADJUSTMENT"
  referenceId Int
  note        String?
  createdAt   DateTime      @default(now())

```

```
    createdBy    Int
    @@map("stock_movements")
}

// ——— AUDIT LOG —————
model AuditLog {
    id            Int          @id @default(autoincrement())
    tenantId      Int
    action        String      // "ORDER_CREATED", "PRODUCT_UPDATED", etc.
    entityType     String      // "Order", "Product", "Invoice"
    entityId      Int
    oldValue      Json?
    newValue      Json?
    userId        Int
    ipAddress     String?
    createdAt     DateTime    @default(now())
    @@map("audit_logs")
}
```

4. Database Relationships

Relationships define how tables connect to each other. Getting relationships right is the most important structural decision you will make. Wrong relationships mean broken queries, missing data, and impossible reports.

4.1 One-to-Many (Most Common in ERP)

One record in Table A relates to many records in Table B. The foreign key always lives on the "many" side.

```
// One Customer → Many Orders
model Customer {
  id      Int      @id @default(autoincrement())
  name    String
  orders  Order[] // The "one" side — no foreign key here
}

model Order {
  id          Int      @id @default(autoincrement())
  customerId  Int      // Foreign key lives on the "many" side
  customer    Customer @relation(fields: [customerId], references: [id])
}

// Query: Get all orders for a customer:
const customerWithOrders = await prisma.customer.findUnique({
  where: { id: 1 },
  include: { orders: true },
});
```

4.2 One-to-One

One record in Table A relates to exactly one record in Table B. Use `@unique` on the foreign key to enforce this.

```
// One Order → One Invoice (an order has exactly one invoice)
model Order {
  id      Int      @id @default(autoincrement())
  invoice Invoice? // Optional — not all orders have invoices yet
}

model Invoice {
  id      Int      @id @default(autoincrement())
  orderId Int      @unique // @unique enforces the one-to-one relationship
  order   Order    @relation(fields: [orderId], references: [id])
}
```

4.3 Many-to-Many (via Join Table)

Many records in Table A relate to many records in Table B. Always use an explicit join table (not Prisma's implicit many-to-many) when you need to store extra data on the relationship.

```
// One Order can have many Products, one Product can be in many Orders
// The join table (OrderItem) stores extra data: quantity, price at time of sale

model Order {
  items OrderItem[]
}

model Product {
  orderItems OrderItem[]
}

model OrderItem {
  id          Int          @id @default(autoincrement())
  orderId     Int
  order       Order        @relation(fields: [orderId], references: [id])
  productId   Int
  product     Product      @relation(fields: [productId], references: [id])
  quantity    Int
  unitPrice   Decimal @db.Decimal(12, 2) // ALWAYS store price at time of order
}
```

 **CRITICAL DESIGN RULE:** ALWAYS store the price in OrderItem at the time of the order. NEVER reference the current product price to calculate historical totals. Product prices change. Historical orders must be immutable.

4.4 Cascade Behavior — What Happens on Delete

When you delete a parent record, what should happen to the child records? In ERP, the wrong cascade setting can silently delete financial data.

Cascade Option	Behavior & When to Use
onDelete: Cascade	Delete all child records when parent is deleted. Use ONLY for items tightly owned by parent (OrderItem when Order is deleted).
onDelete: Restrict (default)	PREVENT deleting the parent if children exist. Use for: cannot delete a Product that has been ordered.
onDelete: SetNull	Set the foreign key to NULL when parent is deleted. Use when the relationship is optional.
onDelete: NoAction	Database does nothing — your app must handle it. Avoid unless you have a specific reason.

 **ERP RULE:** In ThunderERP, we almost never cascade-delete. We soft-delete. A "deleted" customer still has all their order history. Use Restrict on financial relationships to prevent accidental data loss.

4.5 The include vs select Pattern

When querying related data, choose carefully between include (gets everything) and select (gets only what you need). In a production ERP, sending too much data slows down responses.

```
// include - fetches entire related object (use for small relations):
const order = await prisma.order.findUnique({
  where: { id: 1 },
  include: {
    customer: true,      // All customer fields
    items: {
      include: { product: true } // Nested include
    }
  }
});

// select - fetches only specific fields (use for list views/APIs):
const orders = await prisma.order.findMany({
  select: {
    id: true,
    status: true,
    totalAmount: true,
    customer: { select: { name: true, phone: true } }, // Only name and phone
    _count: { select: { items: true } }, // Count of items without fetching them
  }
});
```

5. Migration Discipline — The Non-Negotiable Rules

A migration is a version-controlled record of every change made to your database schema. Migrations are what allow the entire team to have the same database structure, and what allow safe deployments to production.

 **ABSOLUTE RULE:** NEVER modify the database directly through pgAdmin, DBeaver, or raw SQL in development. ALWAYS use Prisma migrations. Direct changes are invisible to the rest of the team and will cause production deployment failures.

5.1 Migration Commands

Command	Purpose & When to Use
<code>npx prisma migrate dev --name <name></code>	Development: Creates a new migration file AND applies it. Use after every schema change.
<code>npx prisma migrate deploy</code>	Production ONLY: Applies all pending migrations without creating new ones.
<code>npx prisma migrate status</code>	Check which migrations have been applied and which are pending.
<code>npx prisma migrate reset</code>	DANGER: Drops all tables, re-runs all migrations from scratch. Development only.
<code>npx prisma migrate diff</code>	Preview what SQL will be generated by the current schema changes.
<code>npx prisma db push</code>	Prototype only: Push schema without a migration file. Never use in team settings.

5.2 Migration Naming Convention

Always name migrations descriptively. The name is permanent — it cannot be changed after other team members pull it.

```
# Good names:
npx prisma migrate dev --name add_products_table
npx prisma migrate dev --name add_gst_percentage_to_products
npx prisma migrate dev --name add_supplier_gst_number
npx prisma migrate dev --name create_audit_logs_table

# Bad names (do not use):
npx prisma migrate dev --name update
npx prisma migrate dev --name fix
npx prisma migrate dev --name changes
```

5.3 What a Migration File Looks Like

Prisma generates SQL migration files in `prisma/migrations/`. Each file contains the actual SQL that was applied. Never edit these files manually.

```
-- prisma/migrations/20240115120000_add_products_table/migration.sql
```

```
CREATE TABLE "products" (  
  "id" SERIAL NOT NULL,  
  "tenantId" INTEGER NOT NULL,  
  "name" VARCHAR(200) NOT NULL,  
  "price" DECIMAL(12,2) NOT NULL,  
  "isActive" BOOLEAN NOT NULL DEFAULT true,  
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT "products_pkey" PRIMARY KEY ("id")  
);  
  
ALTER TABLE "products" ADD CONSTRAINT "products_tenantId_fkey"  
  FOREIGN KEY ("tenantId") REFERENCES "tenants"("id")  
  ON DELETE RESTRICT ON UPDATE CASCADE;
```

5.4 Migration Workflow for Teams

5. Developer A makes a schema change → runs `prisma migrate dev --name description`.
6. Migration file is committed to git along with the schema change.
7. Developer B pulls the changes → runs `npx prisma migrate dev` to apply the new migration.
8. CI/CD pipeline runs `npx prisma migrate deploy` before deploying to production.

 **GIT RULE:** Always commit your `schema.prisma` file and the generated migration files together in the same commit. Never commit one without the other.

6. Prisma Queries — The Complete Reference

6.1 CRUD Operations

Create

```
// Create one record:
const product = await prisma.product.create({
  data: {
    name: "Widget Pro",
    price: 499.99,
    gstPercentage: 18.00,
    tenantId: 1,
    createdBy: userId,
  }
});

// Create with related data (nested create):
const order = await prisma.order.create({
  data: {
    customerId: 5,
    tenantId: 1,
    totalAmount: 999.98,
    createdBy: userId,
    items: {
      create: [ // Create multiple OrderItems in one query
        { productId: 1, quantity: 2, unitPrice: 499.99, gstPct: 18 },
        { productId: 3, quantity: 1, unitPrice: 0.00, gstPct: 5 },
      ]
    }
  },
  include: { items: true } // Return the created items
});

// Create many at once:
await prisma.product.createMany({
  data: [
    { name: "Item A", price: 100, tenantId: 1, createdBy: 1, gstPercentage: 18 },
    { name: "Item B", price: 200, tenantId: 1, createdBy: 1, gstPercentage: 12 },
  ],
  skipDuplicates: true,
});
```

Read

```
// Find unique (throws if not found):
const product = await prisma.product.findUniqueOrThrow({ where: { id: 1 } });

// Find unique (returns null if not found):
const product = await prisma.product.findUnique({ where: { id: 1 } });
```

```
// Find first matching:
const product = await prisma.product.findFirst({
  where: { sku: "WGT-001", isActive: true }
});

// Find many (list with filters, sort, pagination):
const products = await prisma.product.findMany({
  where: {
    isActive: true,
    tenantId: currentTenantId,
    price: { gte: 100, lte: 1000 }, // Between 100 and 1000
    name: { contains: "widget", mode: "insensitive" }, // Case-insensitive search
  },
  orderBy: [{ name: "asc" }],
  skip: (page - 1) * pageSize,
  take: pageSize,
  include: { inventory: true },
});

// Count:
const total = await prisma.product.count({
  where: { isActive: true, tenantId: 1 }
});

// Aggregate:
const stats = await prisma.orderItem.aggregate({
  where: { order: { status: "COMPLETED" } },
  _sum: { unitPrice: true },
  _avg: { quantity: true },
  _count: { id: true },
});
```

Update

```
// Update one record:
const updated = await prisma.product.update({
  where: { id: 1 },
  data: {
    price: 549.99,
    updatedBy: userId,
  }
});

// Atomic increment/decrement (SAFE for stock operations):
await prisma.inventory.update({
  where: { productId: 1 },
  data: {
    availableQty: { decrement: 5 }, // Atomic - safe from race conditions
    reservedQty: { increment: 5 },
  }
});
```

```
// Update many records at once:
await prisma.product.updateMany({
  where: { category: "Electronics", tenantId: 1 },
  data: { gstPercentage: 18 },
});

// Upsert (create if not exists, update if exists):
await prisma.inventory.upsert({
  where: { productId: 1 },
  create: { productId: 1, availableQty: 100 },
  update: { availableQty: { increment: 100 } },
});
```

Soft Delete (Never Hard Delete)

```
// Soft delete – the ONLY acceptable delete in ThunderERP:
await prisma.product.update({
  where: { id: 1 },
  data: {
    isActive: false,
    deletedAt: new Date(),
    deletedBy: userId,
  }
});

// ALWAYS filter isActive in all findMany queries:
where: { isActive: true, tenantId: currentTenantId }
```

6.2 Filter Operators — Full Reference

Operator	Meaning	Example
equals / direct	Exact match	status: "PENDING" or status: { equals: "PENDING" }
not	Not equal	status: { not: "CANCELLED" }
in	Matches any in array	status: { in: ["PENDING","CONFIRMED"] }
notIn	Matches none in array	status: { notIn: ["CANCELLED"] }
gt	Greater than	price: { gt: 0 }
gte	Greater than or equal	quantity: { gte: 1 }
lt	Less than	price: { lt: 1000 }
lte	Less than or equal	stock: { lte: reorderLevel }
contains	String contains	name: { contains: "pro", mode: "insensitive" }
startsWith	String starts with	sku: { startsWith: "WGT" }

endsWith	String ends with	email: { endsWith: "@gmail.com" }
AND	All conditions true	AND: [{ isActive: true }, { price: { gt: 0 } }]
OR	Any condition true	OR: [{ name: { contains: "x" } }, { sku: "ABC" }]
NOT	Negates condition	NOT: { status: "CANCELLED" }
some	At least one related record matches	orders: { some: { status: "PENDING" } }
every	All related records match	items: { every: { quantity: { gt: 0 } } }
none	No related records match	orders: { none: { status: "PENDING" } }

6.3 Pagination Pattern

```
// src/common/dto/pagination.dto.ts
export class PaginationDto {
  @IsOptional() @IsInt() @Min(1)
  page?: number = 1;

  @IsOptional() @IsInt() @Min(1) @Max(100)
  limit?: number = 20;
}

// In service:
async findAll(dto: PaginationDto, tenantId: number) {
  const { page = 1, limit = 20 } = dto;
  const skip = (page - 1) * limit;

  // Use $transaction to get data AND count in a single round-trip:
  const [items, total] = await this.prisma.$transaction([
    this.prisma.product.findMany({
      where: { isActive: true, tenantId },
      skip,
      take: limit,
      orderBy: { createdAt: "desc" },
    }),
    this.prisma.product.count({ where: { isActive: true, tenantId } }),
  ]);

  return {
    data: items,
    meta: {
      total,
      page,
      limit,
      totalPages: Math.ceil(total / limit),
      hasNextPage: page * limit < total,
    }
  }
}
```



```
};  
}
```

7. Transactions — The Most Critical System

A transaction is a group of database operations that must ALL succeed or ALL fail together. In an ERP that handles money and inventory, transactions are not optional — they are the difference between a system that works and one that silently corrupts data.

 **RULE:** Any operation that touches more than one table **MUST** use a transaction. No exceptions. If your service has two or more `prisma.x.create/update` calls, they must be inside `prisma.$transaction()`.

7.1 ACID Properties — What Transactions Guarantee

Property	Meaning & ERP Example
Atomicity	All operations succeed or none do. Order creation + stock deduction happen together or not at all.
Consistency	The DB moves from one valid state to another. Stock cannot go negative. Totals must match line items.
Isolation	Concurrent transactions do not interfere. Two users buying the last stock unit — only one succeeds.
Durability	Once committed, the data survives server crashes, power cuts, and restarts.

7.2 The Standard Transaction Pattern

```
// The complete order creation transaction:
async createOrder(dto: CreateOrderDto, userId: number) {
  return this.prisma.$transaction(async (tx) => {
    // — STEP 1: Validate all stock availability BEFORE creating anything
    for (const item of dto.items) {
      const inventory = await tx.inventory.findUniqueOrThrow({
        where: { productId: item.productId }
      });
      if (inventory.availableQty < item.quantity) {
        throw new BadRequestException(
          `Insufficient stock for product ID ${item.productId}`
        );
      }
    }

    // — STEP 2: Calculate totals
    const products = await tx.product.findMany({
      where: { id: { in: dto.items.map(i => i.productId) } }
    });
    const productMap = new Map(products.map(p => [p.id, p]));
    let totalAmount = 0;
    for (const item of dto.items) {
```

```

    const product = productMap.get(item.productId);
    totalAmount += Number(product.price) * item.quantity;
  }

  // — STEP 3: Create the order with items
  const order = await tx.order.create({
    data: {
      customerId: dto.customerId,
      tenantId: dto.tenantId,
      totalAmount,
      createdBy: userId,
      items: {
        create: dto.items.map(item => ({
          productId: item.productId,
          quantity: item.quantity,
          unitPrice: productMap.get(item.productId).price,
          gstPct: productMap.get(item.productId).gstPercentage,
        }))
      }
    }
  });

  // — STEP 4: Deduct stock atomically
  for (const item of dto.items) {
    await tx.inventory.update({
      where: { productId: item.productId },
      data: { availableQty: { decrement: item.quantity } }
    });
  }

  // — STEP 5: Record stock movements
  await tx.stockMovement.createMany({
    data: dto.items.map(item => ({
      productId: item.productId,
      quantity: item.quantity,
      type: "OUT",
      referenceType: "ORDER",
      referenceId: order.id,
      createdBy: userId,
    }))
  });

  // — STEP 6: Write audit log
  await tx.auditLog.create({
    data: {
      tenantId: dto.tenantId,
      action: "ORDER_CREATED",
      entityType: "Order",
      entityId: order.id,
      newValue: { orderId: order.id, totalAmount },
      userId,
    }
  });

```

```
    return order;
    // If ANY step above throws, ALL steps are rolled back automatically.
  });
}
```

7.3 Transaction Timeout

By default, Prisma transactions time out after 5 seconds. For complex operations, you can extend this:

```
await this.prisma.$transaction(async (tx) => {
  // ... complex operations
}, {
  maxWait: 5000,    // Max time to wait for a transaction slot (ms)
  timeout: 15000,   // Max time the transaction can run (ms)
});
```

7.4 What Happens Without a Transaction

Scenario	Without Transaction	With Transaction
Create order + deduct stock	Order created. Server crashes. Stock never deducted. Overselling possible.	Both succeed or both fail. System is always consistent.
Record payment + update invoice status	Payment recorded. Invoice still shows UNPAID. Customer wrongly billed again.	Both update atomically. Invoice immediately shows PAID.
Create invoice + debit ledger	Invoice exists. No ledger entry. Financial reports are wrong.	Both entries created together or neither.

8. Data Integrity — Golden Constraints

Data integrity means the data in your database is always accurate, consistent, and trustworthy. In a financial ERP, integrity failures are not just bugs — they are accounting errors.

8.1 Constraint Types

Constraint Type	How to Apply & Purpose
Primary Key	@id — Every table must have one. Uniquely identifies every row.
Foreign Key	@relation — Prevents orphan records. Cannot create an Order for a non-existent Customer.
Unique Constraint	@unique — Single field. @@unique([fieldA, fieldB]) — composite unique.
Not Null	Fields without ? are required. Prisma enforces this at the type level.
Default Values	@default(value) — Prevents null values in required fields.
Check Constraints	Use raw SQL in migrations for business rules like stock >= 0.
Index	@@index([field]) — Speeds up queries. Required on all foreign keys and frequently filtered fields.

8.2 Adding Check Constraints (PostgreSQL Level)

Prisma does not natively support CHECK constraints. Add them in a custom migration file. These constraints enforce rules at the database level — even if your application code has a bug.

```
-- Add to a migration file manually:

-- Stock can never be negative:
ALTER TABLE "inventory"
  ADD CONSTRAINT "inventory_available_qty_non_negative"
  CHECK ("availableQty" >= 0);

-- Price must be positive:
ALTER TABLE "products"
  ADD CONSTRAINT "products_price_positive"
  CHECK ("price" > 0);

-- Order total must be positive:
ALTER TABLE "orders"
  ADD CONSTRAINT "orders_total_positive"
  CHECK ("totalAmount" >= 0);

-- Payment amount must be positive:
ALTER TABLE "payments"
  ADD CONSTRAINT "payments_amount_positive"
  CHECK ("amount" > 0);
```

8.3 Indexes — Performance and Integrity

An index speeds up read queries on the indexed column. Without indexes, PostgreSQL scans every row in the table for every query. In a table with 1 million orders, this is catastrophically slow.

```
// In Prisma schema - add indexes to:
// 1. All foreign keys
// 2. Fields you filter by frequently
// 3. Fields you sort by frequently

model Order {
  id          Int          @id @default(autoincrement())
  tenantId    Int
  customerId  Int
  status      OrderStatus
  createdAt   DateTime @default(now())

  @@index([tenantId])           // Always filter by tenant
  @@index([customerId])        // Frequently look up orders by customer
  @@index([status])            // Filter by status often
  @@index([tenantId, status])   // Composite: filter by tenant AND status
  @@index([createdAt])         // Sort by date
}

// UNIQUE indexes (also enforce uniqueness):
model Product {
  sku         String? @unique           // Single column unique

  // Composite unique - sku must be unique per tenant:
  @@unique([tenantId, sku])
}
```

8.4 ERP-Critical Integrity Rules

RULE 1 — Stock Never Goes Negative: Enforce with a CHECK constraint at DB level AND a check in the service before any stock deduction. Belt and suspenders.

RULE 2 — Order Total = Sum of Line Items: Calculate the total in the service. Store it on the Order for query performance. Never recalculate from line items in real-time — prices change.

RULE 3 — Posted Invoices Are Immutable: Once an invoice status is PAID, the invoice record cannot be edited. Enforce in the service: if invoice.status === PAID, throw ForbiddenException.

RULE 4 — No Orphan Financial Records: Every Payment must have an Invoice. Every LedgerEntry must have a Payment. Use Restrict (default) cascade to prevent deletion of parents that have children.

9. Concurrency & Database Locking

Concurrency problems happen when two users perform the same operation at the exact same millisecond. Without protection, your stock levels will go negative and your financial records will be corrupted.

9.1 The Race Condition Problem

```
// Timeline without locking:
// T=0ms: User A reads inventory: availableQty = 1
// T=1ms: User B reads inventory: availableQty = 1
// T=2ms: User A places order for 1 unit - UPDATE inventory SET qty = qty - 1
// T=3ms: User B places order for 1 unit - UPDATE inventory SET qty = qty - 1
// Result: qty = -1 ← CORRUPTED DATA
```

9.2 Solution A — Atomic updateMany with WHERE Guard

This is the simplest and most Prisma-friendly solution. The WHERE clause acts as an optimistic lock.

```
// Only update IF there is enough stock - atomic check + update:
const result = await this.prisma.inventory.updateMany({
  where: {
    productId: productId,
    availableQty: { gte: requestedQty }, // Only update if enough stock exists
  },
  data: {
    availableQty: { decrement: requestedQty }
  }
});

if (result.count === 0) {
  // Either product not found OR stock was just taken by another user
  throw new ConflictException("Stock unavailable. Please try again.");
}
```

9.3 Solution B — SELECT FOR UPDATE (Pessimistic Locking)

For complex multi-step operations, use raw SQL within a transaction to lock the rows you are about to modify.

```
return this.prisma.$transaction(async (tx) => {
  // Lock the inventory row - no other transaction can modify it until this
  // commits:
  const inventory = await tx.$queryRaw`
    SELECT * FROM "inventory"
    WHERE "productId" = ${productId}
    FOR UPDATE
  `;

  if (inventory[0].availableQty < requestedQty) {
```

```
        throw new BadRequestException("Insufficient stock");
    }

    // Safe to update - row is locked:
    await tx.inventory.update({
        where: { productId },
        data: { availableQty: { decrement: requestedQty } }
    });
});
```

Approach	Best For
Atomic updateMany + WHERE guard	Simple stock deductions. Fast, no explicit locking, handles most cases.
SELECT FOR UPDATE (pessimistic)	Complex multi-step operations where you read then write multiple times.
Optimistic locking with version field	High-read, low-write scenarios where conflicts are rare.

10. Soft Deletes & Audit Trails

10.1 Soft Delete Pattern

In ThunderERP, data is never permanently deleted. Financial records, order history, customer data — all of it must be preserved for auditing, reporting, and legal compliance. Soft delete marks records as inactive without removing them.

```
// The soft delete service method:
async softDelete(id: number, userId: number) {
  const record = await this.prisma.product.findUniqueOrThrow({ where: { id } });

  // Check: can we delete this? (business rule)
  const hasActiveOrders = await this.prisma.orderItem.count({
    where: { productId: id, order: { status: { notIn: ["COMPLETED", "CANCELLED"] } } }
  });

  if (hasActiveOrders > 0) {
    throw new ConflictException("Cannot delete product with active orders");
  }

  return this.prisma.$transaction(async (tx) => {
    // Soft delete:
    await tx.product.update({
      where: { id },
      data: { isActive: false, deletedAt: new Date(), deletedBy: userId }
    });
    // Audit log:
    await tx.auditLog.create({
      data: {
        tenantId: record.tenantId,
        action: "PRODUCT_DELETED",
        entityType: "Product",
        entityId: id,
        oldValue: { isActive: true },
        newValue: { isActive: false },
        userId,
      }
    });
  });
}
```

10.2 Prisma Middleware for Auto-filtering

Prisma middleware can automatically add `isActive: true` to every `findMany` and `findFirst` query so you never accidentally return soft-deleted records.

```
// In PrismaService constructor:
this.$use(async (params, next) => {
  // Auto-add soft delete filter:
```

```

    if (params.action === "findMany" || params.action === "findFirst") {
      params.args = params.args || {};
      params.args.where = {
        ...params.args.where,
        isActive: true,
      };
    }
    return next(params);
  });

// NOTE: Use with caution. Explicit where: { isActive: true } is often clearer.
// Only add this middleware after your team agrees on the pattern.

```

10.3 Prisma Extensions — Auto Audit Fields

Prisma Extensions (v4.7+) allow you to intercept all queries and automatically set audit fields like `updatedAt`.

```

// prisma/prisma-extended.service.ts
import { PrismaClient } from "@prisma/client";

export function createExtendedPrisma(userId: number) {
  return new PrismaClient().$extends({
    query: {
      $allOperations: async ({ model, operation, args, query }) => {
        if (["create", "update", "upsert"].includes(operation)) {
          if (args.data) {
            args.data.updatedBy = userId;
            if (operation === "create") args.data.createdBy = userId;
          }
        }
        return query(args);
      }
    }
  });
}

```

11. Raw SQL & Advanced Queries

Sometimes Prisma's query builder is not expressive enough for complex reporting queries. Prisma provides escape hatches to run raw SQL while keeping the rest of your codebase type-safe.

11.1 When to Use Raw SQL

Use Case	Why Raw SQL
Complex aggregation reports	GROUP BY with HAVING, window functions (ROW_NUMBER, LAG, LEAD)
Full-text search	PostgreSQL tsvector/tsquery search
CHECK constraints in migrations	Prisma schema does not support CHECK constraints
EXPLAIN ANALYZE for debugging	Query performance analysis
Recursive CTEs	Hierarchical data (org charts, bill of materials)

11.2 Raw Query Methods

```
// prisma.$queryRaw - returns typed results, SAFE (uses parameterized queries):
const topProducts = await this.prisma.$queryRaw<{ name: string; total: number }[]>`
  SELECT p.name, SUM(oi.quantity) as total
  FROM "order_items" oi
  JOIN "products" p ON p.id = oi."productId"
  WHERE oi."createdAt" >= ${startDate}
  GROUP BY p.id, p.name
  ORDER BY total DESC
  LIMIT ${limit}
`;

// prisma.$executeRaw - for INSERT/UPDATE/DELETE, returns count:
const count = await this.prisma.$executeRaw`
  UPDATE "inventory"
  SET "availableQty" = 0
  WHERE "availableQty" < 0
`;

// NEVER use $queryRawUnsafe with user input - SQL injection risk:
// BAD: prisma.$queryRawUnsafe(`SELECT * FROM products WHERE name =
"${userInput}"`)
// GOOD: prisma.$queryRaw`SELECT * FROM products WHERE name = ${userInput}`
```

11.3 EXPLAIN ANALYZE — Query Performance Debugging

When a query is slow, use EXPLAIN ANALYZE to see the execution plan. This shows you whether PostgreSQL is using your indexes or doing a slow sequential scan.

```
-- Run in DBeaver or pgAdmin:
EXPLAIN ANALYZE
SELECT * FROM "orders"
WHERE "tenantId" = 1 AND "status" = 'PENDING'
ORDER BY "createdAt" DESC
LIMIT 20;

-- Look for:
-- "Seq Scan" → BAD (scanning all rows) → Add an index
-- "Index Scan" → GOOD (using your index)
-- "Bitmap Heap Scan" → OK for medium-size result sets
-- "cost=0.00..X.XX" → Higher X = slower
```

12. Testing Database Logic

Database logic must be tested. Transactions, constraints, and edge cases (negative stock, concurrent updates, invalid state transitions) are the most important things to test — and also the most commonly skipped.

12.1 Test Database Strategy

Approach	Best For
Separate test database	Simple setup. Run "npx prisma migrate dev" on a test DB. Reset between tests.
Database transactions for rollback	Fast. Each test runs in a transaction that is rolled back after. No cleanup needed.
Testcontainers	Production-identical environment. Starts a real PostgreSQL Docker container per test run.
Prisma mock (@prisma/client mock)	Unit tests. No real DB needed. But does not test SQL queries or constraints.

12.2 Test Setup with Transaction Rollback

```
// test/setup.ts
import { PrismaClient } from "@prisma/client";

let prisma: PrismaClient;

beforeAll(async () => {
  prisma = new PrismaClient({
    datasources: { db: { url: process.env.TEST_DATABASE_URL } }
  });
  await prisma.$connect();
});

afterEach(async () => {
  // Clean up in reverse dependency order:
  await prisma.auditLog.deleteMany();
  await prisma.stockMovement.deleteMany();
  await prisma.orderItem.deleteMany();
  await prisma.order.deleteMany();
  await prisma.inventory.deleteMany();
  await prisma.product.deleteMany();
  await prisma.customer.deleteMany();
  await prisma.tenant.deleteMany();
});

afterAll(async () => {
  await prisma.$disconnect();
});
```

12.3 What to Test

- Transaction rollback: Create an order where the stock deduction fails — verify the order was NOT created.
- Check constraint: Try to update inventory to -1 — verify the database rejects it.
- Unique constraint: Try to create two products with the same SKU — verify the second throws.
- Soft delete filter: Create a product, soft-delete it, query findAll — verify it does not appear.
- Concurrency: Simulate two simultaneous requests for the last unit of stock — verify only one succeeds.
- State machine: Try to mark a PENDING order as DELIVERED — verify it throws.

13. Performance & Indexing

As ThunderERP grows, query performance becomes critical. An ERP system that takes 10 seconds to load the order list is not usable. Performance must be designed in from the start, not added later.

13.1 Index Strategy

The rule of thumb: index any column that appears in a WHERE clause, ORDER BY clause, or JOIN condition that runs on a large table.

```
// Always index these in ThunderERP:
model Order {
  tenantId    Int          // ALWAYS - every query filters by tenant
  customerId  Int          // Frequently: "all orders for this customer"
  status      OrderStatus // Frequently: "all pending orders"
  createdAt   DateTime     // Almost always sorted by date

  @@index([tenantId])
  @@index([tenantId, status]) // Composite: two WHERE conditions together
  @@index([tenantId, customerId])
  @@index([createdAt])
}
```

13.2 N+1 Query Problem

The N+1 problem is one of the most common performance pitfalls with ORMs. It happens when you load a list of items (1 query) then load related data for each item separately (N queries).

```
// BAD - N+1 queries (100 orders = 101 database calls):
const orders = await prisma.order.findMany(); // 1 query
for (const order of orders) {
  const customer = await prisma.customer.findUnique({ // 100 more queries
    where: { id: order.customerId }
  });
}

// GOOD - 1 query with JOIN (always use include):
const orders = await prisma.order.findMany({
  include: { customer: true } // Single JOIN query - 1 database call
});

// BEST for list views - select only what you need:
const orders = await prisma.order.findMany({
  select: {
    id: true,
    status: true,
    totalAmount: true,
    customer: { select: { name: true } },
  }
});
```

```
});
```

13.3 Performance Checklist

Issue	Solution
Slow list queries	Add indexes on WHERE and ORDER BY columns. Use select instead of include.
N+1 queries	Always use include for related data. Never query in a loop.
Full table scans	EXPLAIN ANALYZE → add missing index.
Slow pagination	Use cursor-based pagination for large datasets instead of skip/take.
Heavy joins	Use \$queryRaw with optimized SQL for complex reports.
Too much data returned	Add select to return only needed fields. Never return entire objects in list APIs.

14. Debugging Database Issues

When something goes wrong with data, these are the systematic steps to diagnose and fix it.

14.1 Enable Prisma Query Logging

```
// See every SQL query Prisma generates:
const prisma = new PrismaClient({
  log: [
    { emit: "stdout", level: "query" }, // All SQL queries
    { emit: "stdout", level: "error" }, // Errors
    { emit: "stdout", level: "warn" },  // Warnings
  ],
});

// Or enable via environment variable:
// .env: DATABASE_URL="...?schema=public&connection_limit=5"
// Terminal: DEBUG="prisma:query" npm run start:dev
```

14.2 Common Issues & Solutions

Symptom	Diagnosis & Fix
Wrong or missing data in response	Check the Prisma query WHERE clause. Enable query logging and inspect the SQL.
Migration failed or stuck	Check prisma migrate status. Look in prisma/migrations/ for failed migration files.
Slow queries	Run EXPLAIN ANALYZE in DBeaver. Check for Seq Scan — add missing index.
Inconsistent data (order exists, stock not deducted)	Transaction was not used. Wrap multi-table operations in prisma.\$transaction().
P2002 Unique constraint violation	Duplicate data. Check for existing record before create, or use upsert.
P2025 Record not found	findUnique returned null. Use NotFoundException in service.
P2003 Foreign key constraint failed	Trying to create/update with a non-existent related ID. Validate the related ID first.
"Column does not exist" error	Forgot to run npx prisma generate after schema change.
Data returns deleted records	Missing isActive: true in WHERE clause.
Float precision errors in money calculations	Using Float instead of Decimal. Change to Decimal @db.Decimal(12,2).

14.3 Prisma Error Codes Reference

Error Code	Meaning	How to Handle
------------	---------	---------------

P2002	Unique constraint violated	Catch and throw <code>ConflictException</code> with descriptive message.
P2003	Foreign key constraint failed	Validate that related records exist before the operation.
P2025	Record required but not found	Use <code>NotFoundException</code> . Or check your <code>WHERE</code> clause.
P2016	Query interpretation error	Usually a Prisma query syntax error. Check your query structure.
P1001	Cannot reach database	Check <code>DATABASE_URL</code> , Docker container status, firewall rules.
P1002	Database timeout	Increase timeout or optimize the slow query.

15. Production & Deployment Considerations

Production databases require additional care beyond development practices. Data loss in production is permanent.

15.1 Production Migration Workflow

```
# NEVER run "prisma migrate dev" in production
# "migrate dev" can reset the database in some scenarios

# Correct production migration command:
npx prisma migrate deploy

# Add this to your CI/CD pipeline before starting the app:
# 1. npx prisma migrate deploy
# 2. npm run start:prod
```

15.2 Production Connection Settings

```
// Always use connection pooling in production:
DATABASE_URL="postgresql://user:pass@host:5432/db?schema=public&connection_limit=10&pool_timeout=30"

// Or use PgBouncer (recommended for high traffic):
// DATABASE_URL="postgresql://user:pass@pgbouncer-host:6432/db?pgbouncer=true"
```

15.3 Backup Strategy

Task	How & Frequency
Full database backup	pg_dump thunder_erp > backup_\$(date +%Y%m%d).sql — Daily minimum
Point-in-time recovery	Enable WAL archiving in PostgreSQL config — for zero data loss
Test restores	Monthly: restore a backup to a staging environment and verify data
Pre-migration backup	ALWAYS backup immediately before running any migration in production

15.4 Environment Variables

```
# .env.production (never commit to git):
DATABASE_URL="postgresql://prod_user:StrongPassword@db.your-domain.com:5432/thunder_erp_prod"

# .env.development:
DATABASE_URL="postgresql://admin:devpassword@localhost:5432/thunder_erp_dev"
```

```
# .env.test:
DATABASE_URL="postgresql://admin:devpassword@localhost:5432/thunder_erp_test"

# .gitignore - ALWAYS add:
.env
.env.*
!.env.example # Commit a template with no real values
```

16. Common Mistakes & How to Avoid Them

Mistake	Why It Matters & Correct Approach
Using Float for monetary values	Float stores 99.99 as 99.98999... — breaks all financial calculations. ALWAYS use Decimal <code>@db.Decimal(12, 2)</code> .
Editing the database directly in pgAdmin/DBBeaver	Other team members will not have those changes. Production will fail. ALWAYS use migrations.
Modifying existing migration files	Breaks the migration history for everyone. Once committed, migration files are immutable.
Forgetting <code>npx prisma generate</code>	TypeScript will use stale types. You will get runtime errors that look like bugs in your code.
Hard deleting records	Financial history is permanently lost. Auditing becomes impossible. ALWAYS soft delete.
Not using transactions for multi-table operations	Creates inconsistent state — orders without stock deductions, payments without ledger entries.
Missing <code>isActive: true</code> in queries	Soft-deleted records appear in API responses. ALWAYS filter <code>isActive</code> in every <code>findMany</code> .
No indexes on foreign keys	JOINS become full table scans. Queries slow to seconds as data grows.
Storing prices by reference (joining to <code>product.price</code>)	Product prices change. Historical order totals become wrong. ALWAYS store price in the line item.
No tenant isolation in queries	One tenant sees another tenant's data. ALWAYS add <code>tenantId</code> to every <code>WHERE</code> clause.
Running <code>prisma migrate dev</code> in production	Can trigger a database reset in some scenarios. Use <code>prisma migrate deploy</code> in production.

17. Learning Roadmap

Days 1–3 — Foundation

- Complete full setup: PostgreSQL, DBeaver, Prisma, PrismaService in NestJS.
- Read and understand the complete ThunderERP schema (Section 3.5).
- Run your first migration. Open Prisma Studio. Browse the empty tables.
- Write your first Prisma queries: create, findMany, update. Verify in DBeaver.

Week 1 — Core Skills

- Master all CRUD operations (Section 6.1).
- Understand all relationships with real examples (Section 4).
- Write a complete feature: schema change → migration → service → controller.
- Practice all Prisma filter operators (Section 6.2).
- Implement pagination (Section 6.3).

Week 2 — Integrity & Safety

- Understand ACID properties (Section 7.1).
- Write your first multi-step transaction (Section 7.2).
- Add CHECK constraints in a custom migration (Section 8.2).
- Implement soft delete correctly across all service methods.
- Add proper indexes to your schema (Section 8.3).

Week 3 — Advanced Patterns

- Implement the optimistic locking pattern for stock operations (Section 9).
- Write a full audit log system (Section 10.3).
- Use EXPLAIN ANALYZE to debug a slow query (Section 13.2).
- Write at least one raw SQL query for a complex aggregation (Section 11).

Week 4+ — Professional Level

- Set up a test database and write transaction rollback tests (Section 12).
- Understand Prisma Extensions and implement auto audit fields.
- Study production deployment workflow (Section 15).
- Learn PostgreSQL connection pooling and PgBouncer.
- Deep-dive into query performance: window functions, CTEs, full-text search.

18. Quick Reference — Commands & Patterns

Prisma CLI Cheatsheet

Command	Purpose
npx prisma init	Initialize Prisma in a project (first time only)
npx prisma generate	Regenerate client after schema changes (run every time)
npx prisma migrate dev --name x	Create + apply migration in development
npx prisma migrate deploy	Apply migrations in production
npx prisma migrate status	Check pending migrations
npx prisma migrate reset	DANGER: Drop all data and reset (dev only)
npx prisma studio	Open browser-based DB viewer
npx prisma db pull	Reverse-engineer schema from existing database
npx prisma format	Auto-format schema.prisma file
npx prisma validate	Validate schema without generating

PostgreSQL psql Cheatsheet

Command	Purpose
\l	List all databases
\c dbname	Connect to a database
\dt	List all tables
\d tablename	Describe table structure (columns, types, constraints)
\di	List all indexes
\du	List all users/roles
\x	Toggle expanded output (useful for wide tables)
\q	Quit psql
EXPLAIN ANALYZE SELECT...	Show query execution plan with timing

Decimal Calculation Reference

When working with Decimal values from Prisma, they come back as Decimal objects, not JavaScript numbers. Always convert explicitly.

```
// Prisma returns Decimal objects, not numbers:
```

```
const price = product.price; // This is a Decimal, not a number

// Convert to number for calculations:
const priceNum = Number(product.price);
const total = priceNum * quantity;

// Convert back to Decimal string for storage:
// Prisma accepts plain numbers, strings, or Decimal objects:
data: { totalAmount: total } // Prisma handles the conversion

// For display (always show 2 decimal places):
const display = Number(product.price).toFixed(2); // "499.99"

// NEVER compare Decimal with ===:
// price === 499.99 → might fail
// Number(price) === 499.99 → correct
```


19. Final Mindset — Thinking Like a Data Engineer

The database is not just storage. It is the system backbone, the integrity layer, and the single source of truth. Every other component in ThunderERP — the APIs, the business logic, the frontend UI — is ultimately a temporary view over what lives in the database.

The Three Principles

PRINCIPLE 1 — Integrity First: A fast system with wrong data is worse than a slow system with correct data. Always enforce constraints, always use transactions, always validate before writing.

PRINCIPLE 2 — Data Is Permanent: Code can be redeployed. Database changes cannot easily be undone. Think twice before every migration. Test on development before running on production.

PRINCIPLE 3 — Design for the Future: The schema you write today will store data for years. Design with auditability, multi-tenancy, soft deletes, and extensibility (metadata Json field) built in from day one. Retrofitting these later is expensive.

The Failure Chain

```
Bad schema design
  → Wrong data types
  → Data corruption over time

No constraints
  → Invalid records enter the database
  → Broken reports and calculations

No transactions on multi-table operations
  → Partial writes succeed
  → Inconsistent state that is hard to detect

No indexes
  → Queries slow as data grows
  → System becomes unusable at scale

Hard deletes instead of soft deletes
  → Permanent data loss
  → Audit trail gaps and compliance failures
```

“If the database is right, everything else can be fixed.”